

Vault

we are given this contract instance :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Vault {
    bool public locked;
    bytes32 private password;

    constructor(bytes32 _password) {
        locked = true;
        password = _password;
    }

    function unlock(bytes32 _password) public {
        if (password == _password) {
            locked = false;
        }
    }
}
```

and our task is to set locked to false, we observe the public function `unlock` which opens the vault if we give it the correct password, if the password was public we could read the value simply through the `abi` , but sadly since it is private that is not possible

Ethereum is a public blockchain, this means that every node can store a copy of the full ledger and we can query any data recorded by it including smart contract storage

this mean by knowing the address of the contract we can read the value of any state variable even if it is marked as private, what is left now is locating which slot it is stored in, this is easy because the `evm` stores the variables sequentially and order matters, so for example in our contract it will store `locked` in slot 0 and it will take 1 byte from 32 bytes, now when it tries to store `password` which is 32 bytes, slot 0 have 31 bytes left, so it skips this slot and go to the next one which is slot 1, so the password is stored in slot number 1

we can confirm that by using `cast storage` :

```
ethernaut on ♣ master [!1 +2 ?38 ]× cast storage 0xc861d3b365f8a8681cC66C816c52B988b0C16Cdc 0 --rpc-url $rpc_url
0x0000000000000000000000000000000000000000000000000000000000000001

ethernaut on ♣ master [!1 +2 ?38 ]> cast storage 0xc861d3b365f8a8681cC66C816c52B988b0C16Cdc 1 --rpc-url $rpc_url
0x412076657279207374726f6e67207365637265742070617373776f7264203a29
```

now we send this value to the contract to solve the challenge :

[illegible]

as we can see slot 0 became equal to 0 meaning the value of locked is False

Mission done !!